

Credit Card Approval Prediction: ML-Powered Credit Card Approval System

Project Description:

The Credit Card Approval Prediction system uses Machine Learning to automate the traditionally manual credit card approval process. Applicant details such as income, employment status, education, and financial history are entered through a Flask-based web application. The trained machine learning model analyzes this input data and instantly predicts whether the application should be Approved or Rejected, replacing a slow, error-prone manual review with a fast, consistent, data-driven decision.

The system is built around a complete ML pipeline covering data preprocessing, feature engineering, model training, and model evaluation, with Logistic Regression, Decision Tree, Random Forest, and XGBoost Classifier trained and compared to select the best-performing model. The chosen model is integrated into a Flask web application with an HTML/CSS interface, allowing users to submit applicant details and receive a real-time approval or rejection result. This reduces processing time, improves prediction accuracy, and supports quick, reliable decision-making for banks and financial institutions.

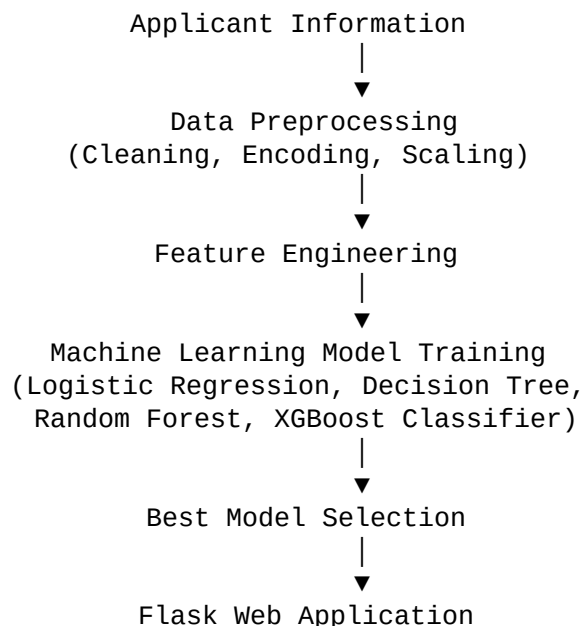
Scenarios

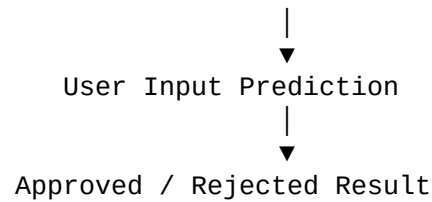
Scenario 1: An applicant with a stable income, permanent employment, and a good education background submits their details through the Flask form. The trained model evaluates the inputs against the patterns learned during training and returns an Approved result in real time, along with a clean confirmation message on the results page.

Scenario 2: An applicant with irregular income and limited employment history submits their application. The model identifies the higher risk profile from the applicant's financial and personal attributes and returns a Rejected result, helping the bank avoid a high-risk approval without manual review.

Scenario 3: A bank official processes a batch of new applications through the same web interface instead of manually verifying each one. Each applicant's details are entered once, and the system returns an instant Approved or Rejected decision for every case, significantly cutting down the time needed compared to the traditional manual verification process.

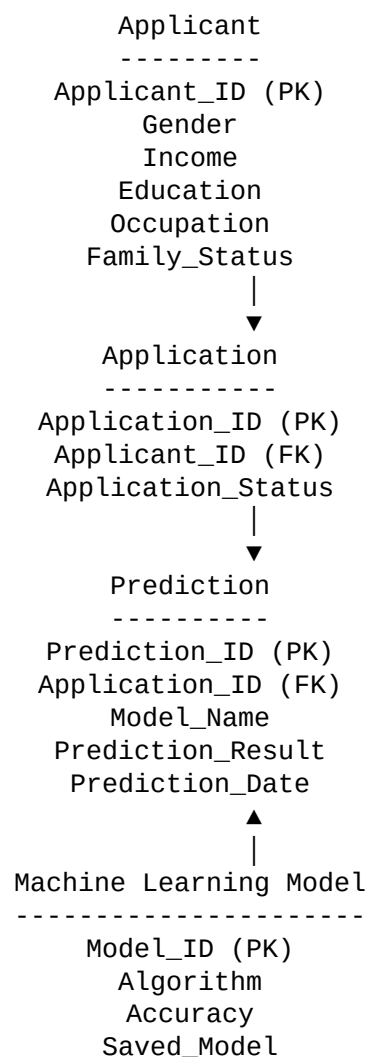
Technical Architecture:





Description: Credit Card Approval Prediction uses a modular pipeline to deliver fast, ML-driven decisions. The Flask-based frontend collects applicant information through a responsive HTML/CSS form, while the backend handles data preprocessing, feature scaling, and loads the saved model to generate a prediction. The best-performing classifier is selected after training and evaluating multiple algorithms, then saved with joblib and served through the Flask application for real-time predictions.

Entity Diagram



Description: The dataset is organized around four related entities. Each Applicant has one or more Applications, each Application is linked to a Prediction generated by a specific Machine Learning Model, and the model's algorithm and accuracy are tracked so the best-performing version can be identified and reused.

Pre-requisites:

- **Python Programming Proficiency:** Python 3.x fundamentals for data processing and model development.
- **Machine Learning Familiarity:** Understanding of classification algorithms such as Logistic Regression, Decision Tree, Random Forest, and XGBoost.
- **Flask Framework Knowledge:** Basics of routing, templates, and handling form submissions in Flask.
- **HTML, CSS Skills:** Building a simple, responsive interface for applicant data entry and result display.
- **Data Analysis Libraries:** Working knowledge of Pandas, NumPy, Matplotlib, and Scikit-learn.
- **Version Control with Git:** Using Git and GitHub for source code management and collaboration.
- **Development Environment Setup:** Google Colab for model training and VS Code for application development.

Requirements

Hardware

- Intel Core i3 or above
- 4GB RAM (Minimum)
- 10GB Free Storage

Software

- Windows 10/11
- Python
- Google Colab
- VS Code
- GitHub

Technologies Used

- Python
- Machine Learning
- Flask
- HTML
- CSS
- Google Colab
- VS Code
- GitHub
- Pandas
- NumPy
- Matplotlib
- Seaborn
- Scikit-learn
- XGBoost
- imbalanced-learn (SMOTE)

Algorithms Used

- Logistic Regression
- Decision Tree
- Random Forest
- XGBoost Classifier

Project Workflow:

Milestone 1: Data Collection and Preprocessing

This milestone focuses on gathering the credit card applicant dataset and preparing it for model training through merging, cleaning, encoding, and feature engineering.

- **Activity 1.1:** Collect two source datasets — application_record.csv (applicant demographic and financial details) and credit_record.csv (monthly credit status history) — and merge them on the shared ID column.
- **Activity 1.2:** Derive the target label from the STATUS column (Approved if status is C, X, or 0; Rejected otherwise), keep only the latest record per customer, and drop identifier columns no longer needed (ID, STATUS, MONTHS_BALANCE).
- **Activity 1.3:** Handle missing values by filling OCCUPATION_TYPE with "Unknown", and engineer AGE and EMPLOYMENT_YEARS from the raw DAYS_BIRTH and DAYS_EMPLOYED columns.
- **Activity 1.4:** Encode every categorical column with a LabelEncoder and save the fitted encoders (encoders.pkl) so the Flask app can apply the exact same encoding to new applicants.

```
# Merge Application & Credit Datasets
data = pd.merge(application, credit, on="ID", how="inner")

# Create Target Variable from STATUS
data["TARGET"] = data["STATUS"].apply(lambda x: 1 if x in ["C", "X", "0"] else 0)

# Keep the latest record per customer
data = data.sort_values(by="MONTHS_BALANCE")
data = data.drop_duplicates(subset="ID", keep="last")
data.drop(["ID", "STATUS", "MONTHS_BALANCE"], axis=1, inplace=True)

# Fill missing values
data["OCCUPATION_TYPE"] = data["OCCUPATION_TYPE"].fillna("Unknown")

# Feature Engineering
data["AGE"] = (-data["DAYS_BIRTH"]) // 365
data["EMPLOYMENT_YEARS"] = (-data["DAYS_EMPLOYED"]) // 365
data.drop(["DAYS_BIRTH", "DAYS_EMPLOYED"], axis=1, inplace=True)

# Encode Categorical Features and Save Encoders
from sklearn.preprocessing import LabelEncoder
import joblib

encoders = {}
categorical_columns = data.select_dtypes(include="object").columns

for column in categorical_columns:
    encoder = LabelEncoder()
    data[column] = encoder.fit_transform(data[column])
    encoders[column] = encoder

joblib.dump(encoders, "encoders.pkl")
```

Milestone 2: Model Training and Evaluation

- **Activity 2.1:** Perform exploratory data analysis (EDA), visualizing the approval distribution, gender split, annual income distribution, education levels, and feature correlations via a heatmap.
- **Activity 2.2:** Split the data into training and test sets (80/20, stratified on the target), then balance the training set with SMOTE, since approved and rejected applications are not evenly distributed.
- **Activity 2.3:** Scale the balanced features with a StandardScaler, then train and compare four classification models: Logistic Regression, Decision Tree, Random Forest, and XGBoost.
- **Activity 2.4:** Evaluate each model with accuracy, and inspect the confusion matrix and classification report for the strongest candidates before selecting the best-performing model.
- **Activity 2.5:** Save the selected model, the fitted StandardScaler, and the LabelEncoders using Joblib, so the Flask app can preprocess new applicants exactly as during training.

Among the four algorithms compared, the Decision Tree Classifier was selected as the best-performing model and saved as `best_model.pkl`, alongside `scaler.pkl` (StandardScaler) and `encoders.pkl`.

```
# Split Features and Target
X = data.drop("TARGET", axis=1)
y = data["TARGET"]

# Train/Test Split (stratified)
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42, stratify=y
)

# Balance the training data using SMOTE
from imblearn.over_sampling import SMOTE
smote = SMOTE(random_state=42)
X_train, y_train = smote.fit_resample(X_train, y_train)

# Feature Scaling
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Train and Compare Models
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from xgboost import XGBClassifier
from sklearn.metrics import accuracy_score

lr = LogisticRegression(max_iter=1000).fit(X_train, y_train)
dt = DecisionTreeClassifier(random_state=42).fit(X_train, y_train)
rf = RandomForestClassifier(n_estimators=100, max_depth=15,
                           random_state=42, n_jobs=-1).fit(X_train, y_train)
xgb = XGBClassifier(n_estimators=100, max_depth=6, learning_rate=0.1,
                    random_state=42, eval_metric="logloss").fit(X_train, y_train)

# Save Best Model
import joblib
joblib.dump(dt, "best_model.pkl")
joblib.dump(scaler, "scaler.pkl")
joblib.dump(encoders, "encoders.pkl")
```

Milestone 3: Flask Application Development (app.py)

This milestone focuses on building app.py, which loads the saved model, scaler, and encoders, and serves real-time predictions.

- **Activity 3.1:** Define the home route to load the saved encoders and pass their class labels to the template, populating every dropdown dynamically.
- **Activity 3.2:** Define the /predict route to read all 17 form fields, encode categorical values with the saved LabelEncoders, scale the feature vector with the saved StandardScaler, and return the model's prediction.

```
from flask import Flask, render_template, request
import joblib
import numpy as np

app = Flask(__name__)

# Load Model, Scaler and Encoders
model = joblib.load("best_model.pkl")
scaler = joblib.load("scaler.pkl")
encoders = joblib.load("encoders.pkl")

@app.route("/")
def home():
    dropdowns = {}
    for col, encoder in encoders.items():
        dropdowns[col] = list(encoder.classes_)
    return render_template("index.html", dropdowns=dropdowns)

@app.route("/predict", methods=["POST"])
def predict():
    try:
        gender = request.form["CODE_GENDER"]
        own_car = request.form["FLAG_OWN_CAR"]
        own_realty = request.form["FLAG_OWN_REALTY"]
        children = int(request.form["CNT_CHILDREN"])
        income = float(request.form["AMT_INCOME_TOTAL"])
        income_type = request.form["NAME_INCOME_TYPE"]
        education = request.form["NAME_EDUCATION_TYPE"]
        family = request.form["NAME_FAMILY_STATUS"]
        housing = request.form["NAME_HOUSING_TYPE"]
        age = int(request.form["AGE"])
        employment = int(request.form["EMPLOYMENT_YEARS"])
        mobile = int(request.form["FLAG_MOBIL"])
        work_phone = int(request.form["FLAG_WORK_PHONE"])
        phone = int(request.form["FLAG_PHONE"])
        email = int(request.form["FLAG_EMAIL"])
        occupation = request.form["OCCUPATION_TYPE"]
        family_members = float(request.form["CNT_FAM_MEMBERS"])

        # Encode categorical values
        gender = encoders["CODE_GENDER"].transform([gender])[0]
        own_car = encoders["FLAG_OWN_CAR"].transform([own_car])[0]
        own_realty = encoders["FLAG_OWN_REALTY"].transform([own_realty])[0]
        income_type = encoders["NAME_INCOME_TYPE"].transform([income_type])[0]
        education = encoders["NAME_EDUCATION_TYPE"].transform([education])[0]
        family = encoders["NAME_FAMILY_STATUS"].transform([family])[0]
        housing = encoders["NAME_HOUSING_TYPE"].transform([housing])[0]
        occupation = encoders["OCCUPATION_TYPE"].transform([occupation])[0]

        features = np.array([gender, own_car, own_realty, children, income,
                             income_type, education, family, housing, age,
```

```
employment, mobile, work_phone, phone, email,
occupation, family_members]])

features = scaler.transform(features)
prediction = model.predict(features)

if prediction[0] == 1:
    result = "\u2705 Credit Card Approved"
else:
    result = "\u274c Credit Card Rejected"

return render_template("result.html", prediction=result)

except Exception as e:
    return str(e)

if __name__ == "__main__":
    app.run(debug=True)
```

Milestone 4: Frontend Development

- **Activity 4.1:** Design a two-panel, glassmorphism-style interface (style.css) with a gradient background, a left panel introducing the system and its key features (Secure Prediction, Instant Results, Machine Learning), and a right panel containing the applicant details form with dropdowns populated dynamically from the saved encoders.
- **Activity 4.2:** Build result.html to clearly display the prediction outcome — a green success state for Approved and a red state for Rejected — with an option to predict another application.

Milestone 5: Deployment

- **Activity 5.1:** Prepare the application for local deployment by creating a virtual environment and a requirements.txt file with all dependencies.
- **Activity 5.2:** Deploy the Flask application to a hosting platform such as Render, Railway, PythonAnywhere, or Docker for public access.

Milestone 6: Conclusion

With the model trained, the Flask backend built, the frontend completed, and the application deployed, the Credit Card Approval Prediction system is ready to deliver fast, consistent, and accurate approval decisions.

Output

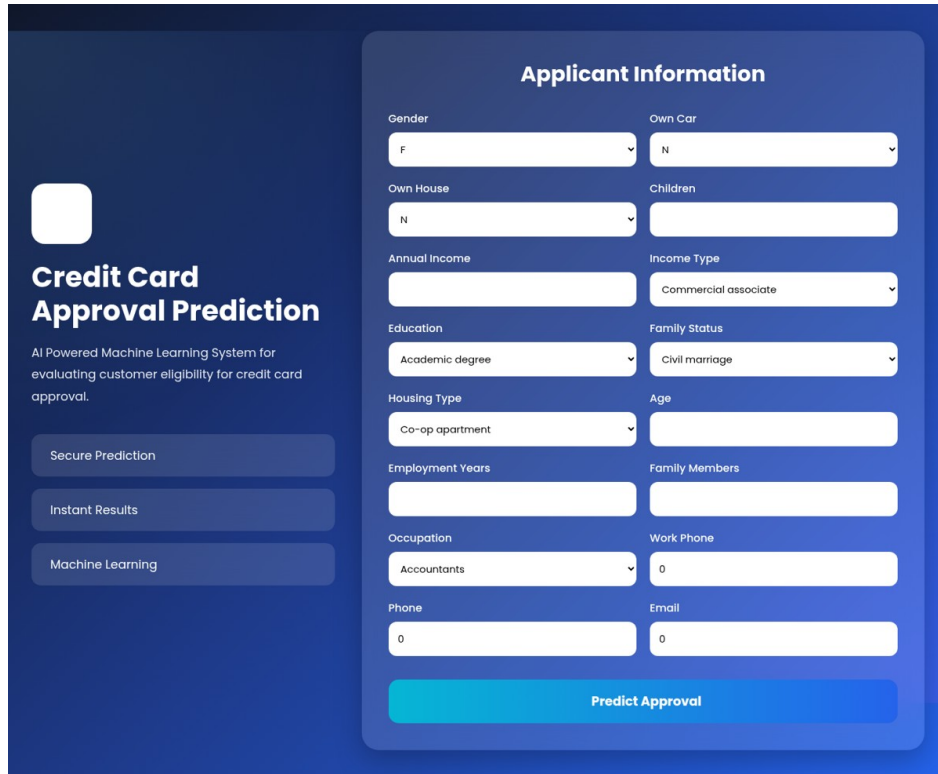
The user enters applicant details in the Flask application. The trained machine learning model predicts:

-  Credit Card Approved
-  Credit Card Rejected

Exploring the Website's Web Pages:

Home / Applicant Information Page:

Description: The home route (/) loads the saved LabelEncoders and passes their class labels into index.html, so every dropdown (Gender, Income Type, Education, Family Status, Housing Type, Occupation) is generated dynamically from the same categories the model was trained on. The page uses a two-panel glassmorphism layout — an introduction panel on the left and the applicant details form on the right.



The screenshot shows a web application interface for "Credit Card Approval Prediction". The interface is divided into two main panels. The left panel, titled "Credit Card Approval Prediction", features a subtitle "AI Powered Machine Learning System for evaluating customer eligibility for credit card approval." and three buttons: "Secure Prediction", "Instant Results", and "Machine Learning". The right panel, titled "Applicant Information", contains a form with various input fields and dropdown menus. The form fields are arranged in two columns. The left column includes: Gender (dropdown with 'F' selected), Own House (dropdown with 'N' selected), Annual Income (text input), Education (dropdown with 'Academic degree' selected), Housing Type (dropdown with 'Co-op apartment' selected), Employment Years (text input), Occupation (dropdown with 'Accountants' selected), Phone (text input with '0' selected), and a "Predict Approval" button. The right column includes: Own Car (dropdown with 'N' selected), Children (text input), Income Type (dropdown with 'Commercial associate' selected), Family Status (dropdown with 'Civil marriage' selected), Age (text input), Family Members (text input), Work Phone (text input with '0' selected), and Email (text input with '0' selected).

Filled Applicant Form:

Description: The applicant enters their details — gender, car/property ownership, income, income type, education, family status, housing type, age, employment years, occupation, and contact flags — before submitting the form to the /predict route.



Credit Card Approval Prediction

AI Powered Machine Learning System for evaluating customer eligibility for credit card approval.

Secure Prediction

Instant Results

Machine Learning

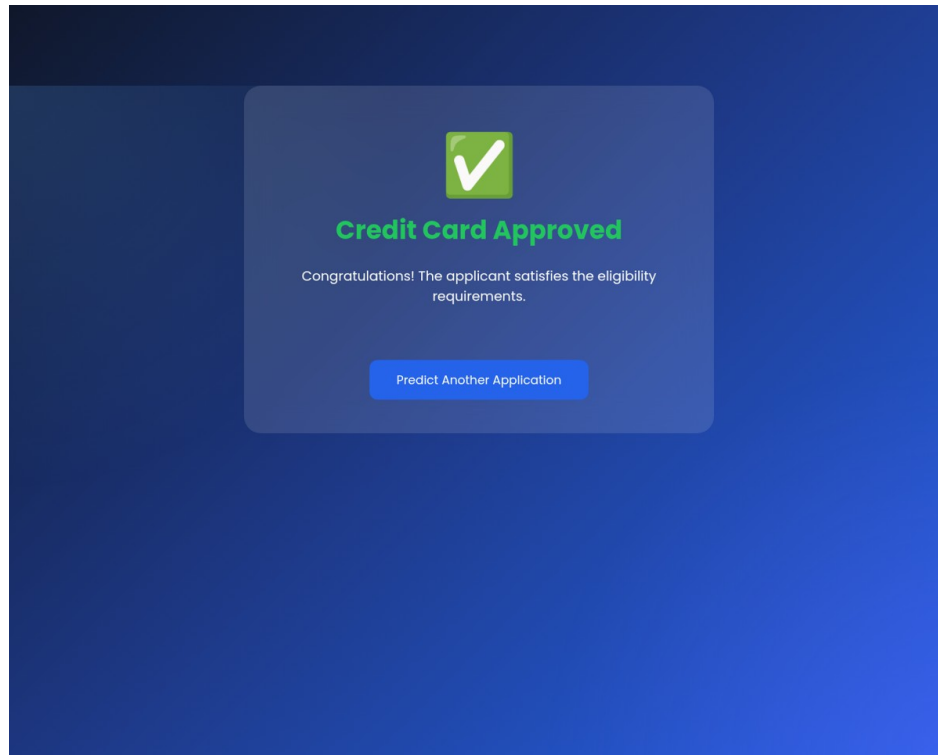
Applicant Information

Gender	Own Car
F	Y
Own House	Children
Y	0
Annual Income	Income Type
250000	Working
Education	Family Status
Higher education	Married
Housing Type	Age
House / apartment	35
Employment Years	Family Members
8	2
Occupation	Work Phone
Managers	1
Phone	Email
1	1

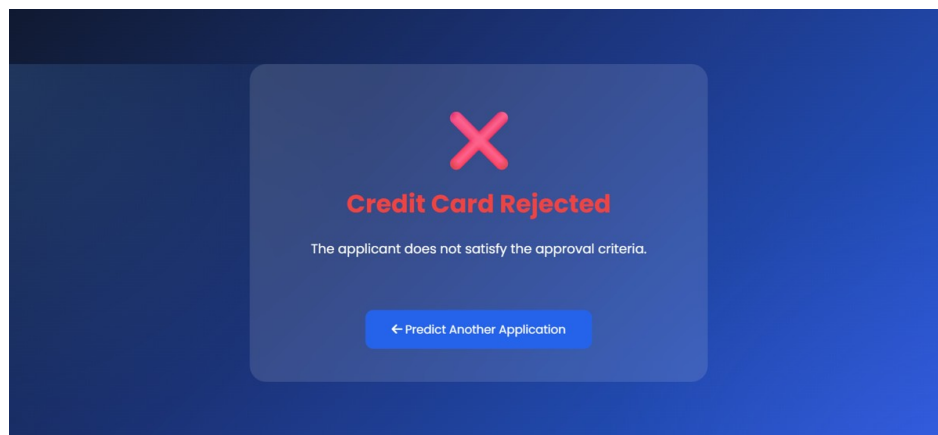
Predict Approval

Result Page:

Description: After submission, result.html displays the prediction outcome returned by the model. An Approved result is shown with a green checkmark and a congratulatory message, while a Rejected result is shown in red with a message explaining that the applicant does not meet the approval criteria. A button lets the user return to the form to predict another application.



Rejected outcome from a live prediction:



Conclusion:

The Credit Card Approval Prediction System successfully automates the credit card approval process using Machine Learning. The system preprocesses applicant data, trains multiple classification models, and selects the best-performing model for prediction. A Flask-based web application provides a simple interface for users to enter applicant information and receive real-time approval or rejection results. This project reduces manual effort, improves decision-making efficiency, and demonstrates the practical application of machine learning in the banking and financial sector.